



Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

КУРСОВА РОБОТА

з кредитного модуля «Методи машинного навчання»

**за темою «Реалізація моделі, що передбачає середню вартість
житла у Каліфорнії»**

Виконала:

студентка групи ФІ-73

Звичайна Анастасія

Олександрівна

залікова книжка №04

Викладач:

Куcssуль Наталія Миколаївна

Київ – 2020

РЕАЛІЗАЦІЯ МОДЕЛІ, ЩО ПЕРЕДБАЧАЄ СЕРЕДНЮ ВАРТІСТЬ ЖИТЛА У КАЛІФОРНІЇ

ЗМІСТ

Вступ.....	4
Розділ 1. Підготовчий етап аналізу.....	5
Розділ 2. Реалізація моделі.....	25
Розділ 3. Аналіз результатів.....	27
Висновок.....	32
Список використаних джерел.....	32

ВСТУП

Зараз методи машинного навчання актуальні як ніколи, адже сьогодні існує дуже велика кількість даних, які людина просто фізично не може обробити через брак часу, а задачі, які потребують аналізу таких даних, виникають. Це можуть бути задачі класифікації, кластеризації чи регресії.

У даній роботі я хочу розглянути один із прикладів задачі регресії – прогнозування середньої вартості житла, в залежності від попередніх даних та 9 різних ознак: довготи, широти, середнього віку житла, загальної кількості кімнат, загальної кількості ванних кімнат, чисельності населення, де цей будинок знаходиться, чисельності людей, що проживають у жилому будинку, середнього доходу інших жильців цього дому та місця розташування будинку відносно виходу до моря чи океану.

Актуальність роботи зумовлена тим, що на даний момент така задача є реальною задачею, що може виникнути у бізнесмена, який хоче вкласти гроші у будівництво того чи іншого будинку, чи у просто у людини, яка хоче самостійно проаналізувати ринок та продати чи купити житло за вигідною для неї вартістю.

Метою дослідження роботи було закріпити вже отримані навички з методів машинного навчання на прикладі побудови на мові програмування Python моделі, що передбачає середню вартість житла у Каліфорнії.

Досягнення мети дослідження: реалізована модель, що передбачає середню вартість житла у Каліфорнії в залежності від попередніх даних та 9 різних ознак.

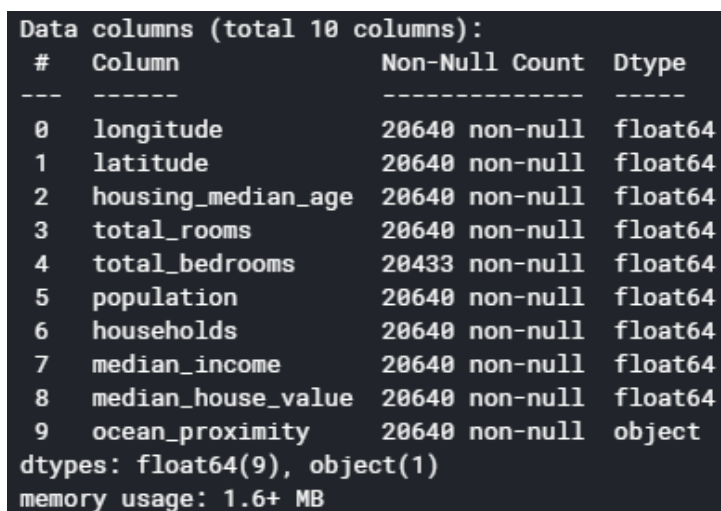
Методика дослідження: наукова, аналітична, моделювання, порівняння та узагальнення.

РОЗДІЛ 1

ПІДГОТОВЧИЙ ЕТАП АНАЛІЗУ

Спочатку треба завантажити обраний датасет з kaggle і це було зроблено ось так: `housing_prices_base=pd.read_csv('./input/california-housing-prices/housing.csv')`

Далі, використовуючи команду `info()` були проглянуті ознаки які є у даному датасеті (див. рис.1): `housing_prices_base.info()`



Data columns (total 10 columns):				
#	Column	Non-Null	Count	Dtype
0	longitude	20640	non-null	float64
1	latitude	20640	non-null	float64
2	housing_median_age	20640	non-null	float64
3	total_rooms	20640	non-null	float64
4	total_bedrooms	20433	non-null	float64
5	population	20640	non-null	float64
6	households	20640	non-null	float64
7	median_income	20640	non-null	float64
8	median_house_value	20640	non-null	float64
9	ocean_proximity	20640	non-null	object

dtypes: float64(9), object(1)
memory usage: 1.6+ MB

Рис.1. Коротка інформація про датасет.

Прогнозуємою ознаку була вибрана середня вартість будинку, тобто `median_house_value`.

1.1. Аналіз набору даних на наявність пропущених значень та NaN.

Бачимо, що у вибраному датасеті є NaN значення у ознаці `total_bedrooms`. Їх кількість мала, порівнюючи з кількістю самих спостережень, тому можемо просто їх видалити, адже це не буде впливати на точність прогнозу. Видалення робимо, використовуючи метод `dropna()` наступним чином:

```
housing_prices_base=housing_prices_base.dropna()
```

Для наочності була зроблена візуалізація того, що вже дані не містять невизначеності, тобто пропущених значень (див. рис.2).

```
import missingno as msno
```

```
msno.bar(housing_prices_base, color = 'b', labels = True)
```

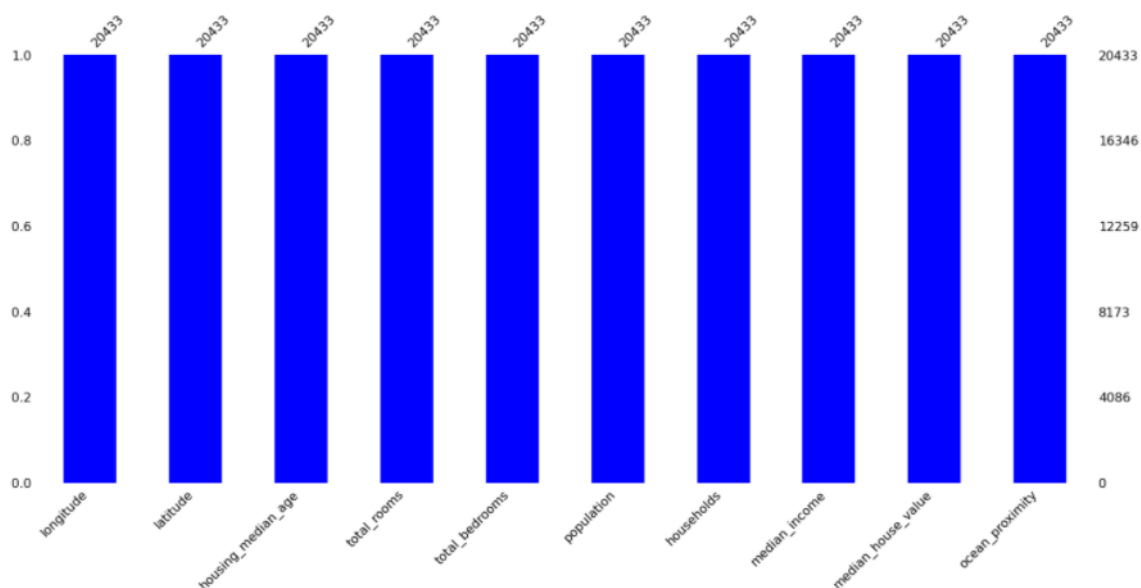


Рис.2. Візуалізація пропущених даних

1.2. Візуалізація даних (Line Plot, Bare Chart, Histogram, Heatmap).

З метою оцінки розподілу даних були побудовані гістограми (див. рис.3), використовуючи метод `hist()` та встановлюючи кількість бінів рівним 50.

```
housing_prices_base.hist(bins=50, figsize=(20,10))
```

```
plt.show()
```

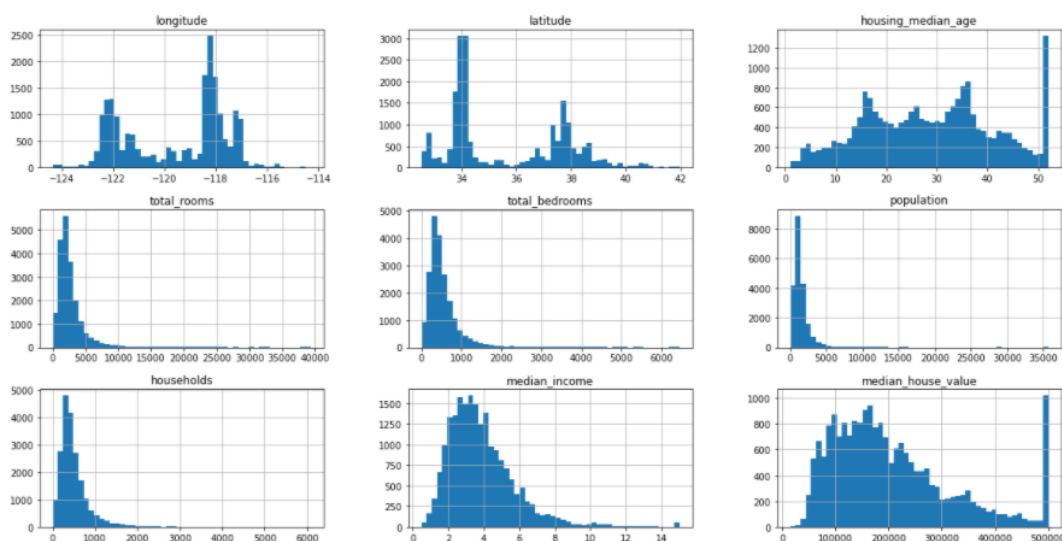


Рис.3. Гістограми розподілів ознак датасету.

Бачимо, що розподіл таких ознак як `total_rooms`, `total_bedrooms`, `population` та `households` схожий а нормальний, але це тільки на перший погляд. Збільшуючи кількість бінів та масштаб можемо переконатись у оберненому, побачити певні викиди (див. рис.4).

```
fig = plt.figure(figsize = (20, 10))

ax_1 = fig.add_subplot(2, 2, 1)

ax_2 = fig.add_subplot(2, 2, 2)

ax_3 = fig.add_subplot(2, 2, 3)

ax_4 = fig.add_subplot(2, 2, 4)

housing_prices_base['total_rooms'].hist(bins=1000, ax=ax_1)

housing_prices_base['total_bedrooms'].hist(bins=1000, ax=ax_2)

housing_prices_base['population'].hist(bins=1000, ax=ax_3)

housing_prices_base['households'].hist(bins=1000, ax=ax_4)

plt.show()
```

Яскраво ми можемо побачити викид ознаки `median_house_value` у районі 500000 доларів (якщо вважаємо номінал ціни у доларах). Такі викиди дуже часто псують точність прогнозу аналітикам і їх зазвичай прибирають, але це не у нашому випадку, адже у результаті різниці точності прогнозу будуть становити не більше 0.01 (я це пов'язую з тим, що таких викидів приблизно 1000, що не є дуже великим числом, порівнюючи з кількістю спостережень у датасеті), тому я не вважаю за необхідним робити певні втручання у датасет і видаляти дані.

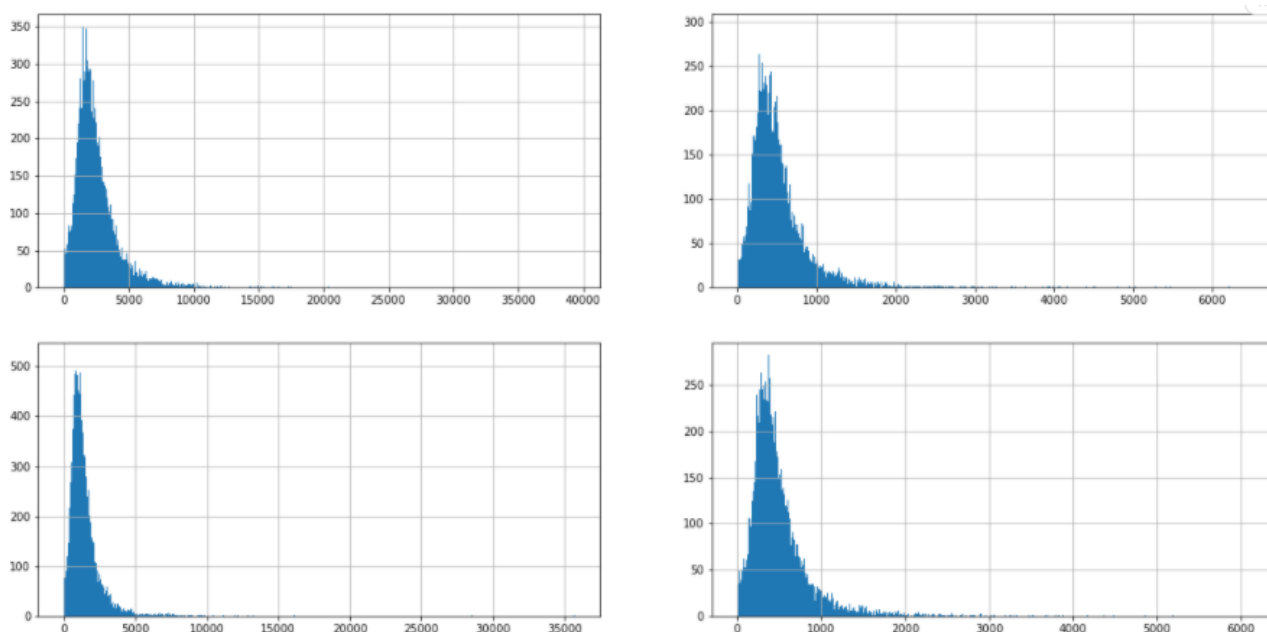


Рис.4. Гістограми розподілів ознак `total_rooms`, `total_bedrooms`, `population` та `households`.

Ми маємо також категоріальну ознаку `ocean_proximity`, візуалізацію (див. рис.5) якої ми можемо зробити наступним чином:

```
sns.countplot(housing_prices_base['ocean_proximity'])
```

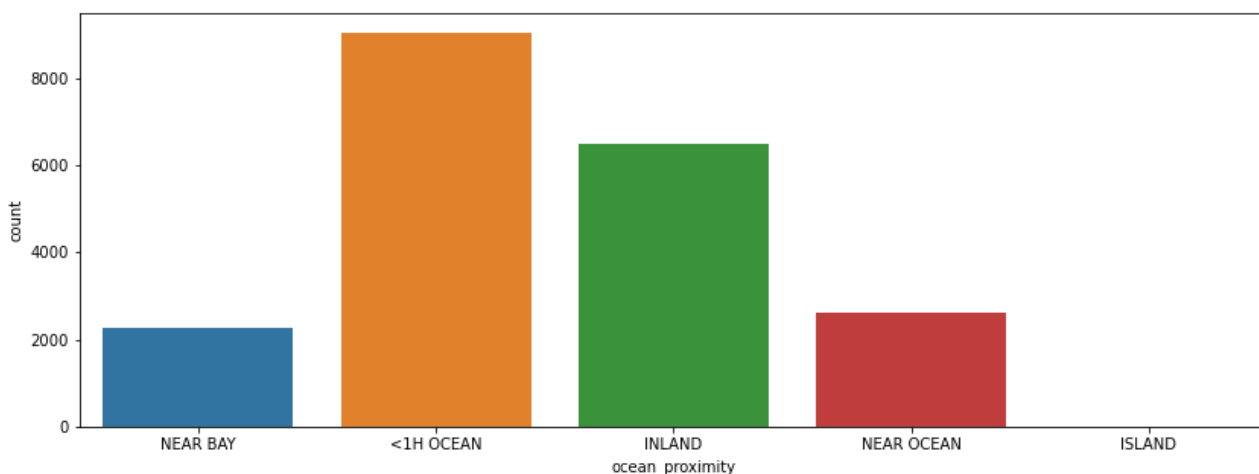


Рис.5. Гістограма ознаки `ocean_proximity`.

Бачимо, що більше всього житла знаходиться саме біля океану, а менше за все на островах, що є очікуваним.

Також ми можемо зробити «датафреймовську» візуалізацію і побачити розподіл по цінам на житло, у залежності від того де знаходиться житло відносно виходу до океану чи затоки (див. рис.6).

```
housing_prices_base.groupby(['ocean_proximity'])['median_house_value',
'median_income'].mean()
```

	median_house_value	median_income
ocean_proximity		
<1H OCEAN	240267.990812	4.231101
INLAND	124896.863147	3.210359
ISLAND	380440.000000	2.744420
NEAR BAY	259279.292070	4.175647
NEAR OCEAN	249042.355023	4.006374

Рис.6. Розподіл по цінам відносно ocean_proximity.

Бачимо, що ціни, на житло з виходом до океану чи затоки вище у два рази, ніж ціни на житло без виходу до них, а найвищі ціни на житло, що знаходиться на островах.

Наступною візуалізацією, що була побудована, є графіки лінійної залежності між набором даних та median_house_value (див. рис.7). Для цього використовувались бібліотеки seaborn та matplotlib наступним чином:

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
col=4
```

```
row=2
```

```
fig, axes = plt.subplots(row, col, figsize=(30, 14))
```

```
for index, feature in enumerate(housing_prices_base.columns[:8]):
```

```
    sns.lineplot(data=housing_prices_base,x=feature,y='median_house_value',ax=axes[
index//4,index%4])
```

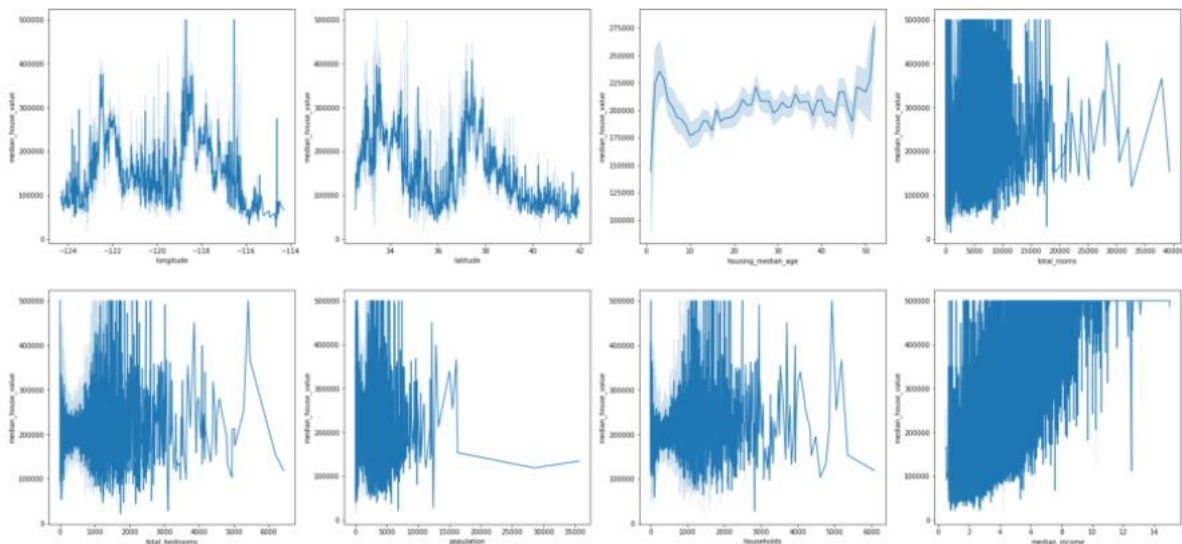


Рис.7. Графіки лінійної залежності.

Очевидно, що яскравої залежності між набором даних та `median_house_value` ми на жаль не спостерігаємо, але бачимо, що найбільше виділяється останній графік: графік залежності `median_house_value` від `median_income`, тобто залежності середньої вартості житла від середньої заробітної плати людей вже заселених будинків того району, де це житло знаходиться. Дана залежність має місце на існування, адже у багатьох країнах можна виділити території, де проживає багата частина населення, а де бідна.

Наступний графік, що був побудований, це `barplot` (див. рис.8), який є не дуже інформативним, адже по суті це є той самий графік, що і у попередньому випадку, просто представлений у іншому вигляді.

```
col=4
```

```
row=2
```

```
fig, axes = plt.subplots(row, col, figsize=(30, 14))
```

```
for index, feature in enumerate(housing_prices_base.columns[:8]):
```

```
    sns.barplot(data=housing_prices_base, x=feature, y='median_house_value', ax=axes[
        index//4, index%4])
```

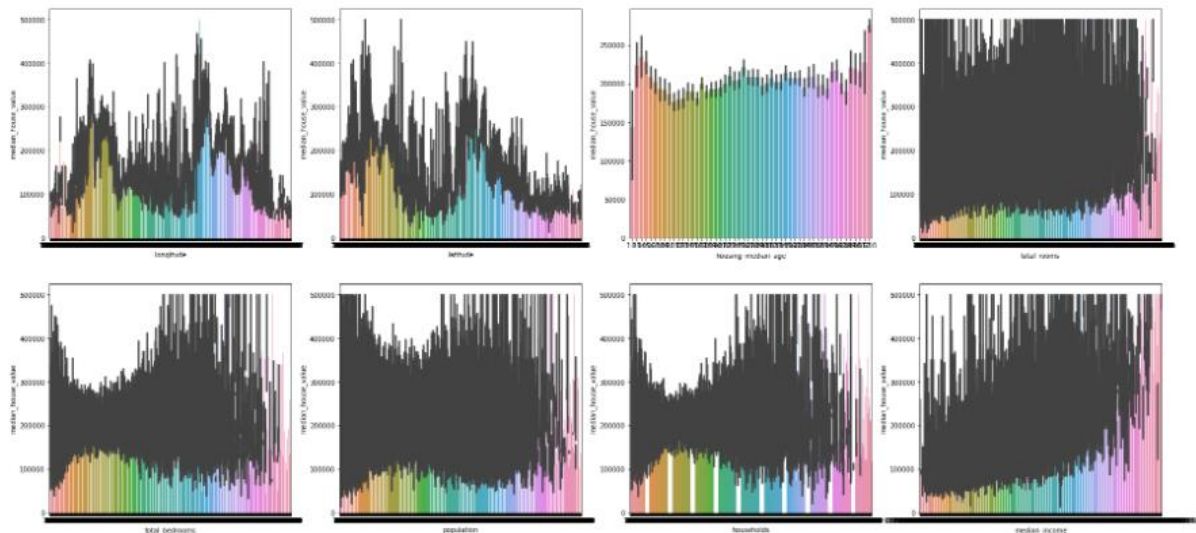


Рис.8. VAR-графіки.

Далі був побудований на мою думку найкращий графік для будь-якого аналізу: heatmap-графік, на якому ми чітко бачимо у числовому еквівалентні де і які залежності можна спостерігати (див. рис.9).

```
fig = plt.figure(figsize=(10,10))
```

```
sns.heatmap(data=housing_prices_base.corr(),annot=True)
```

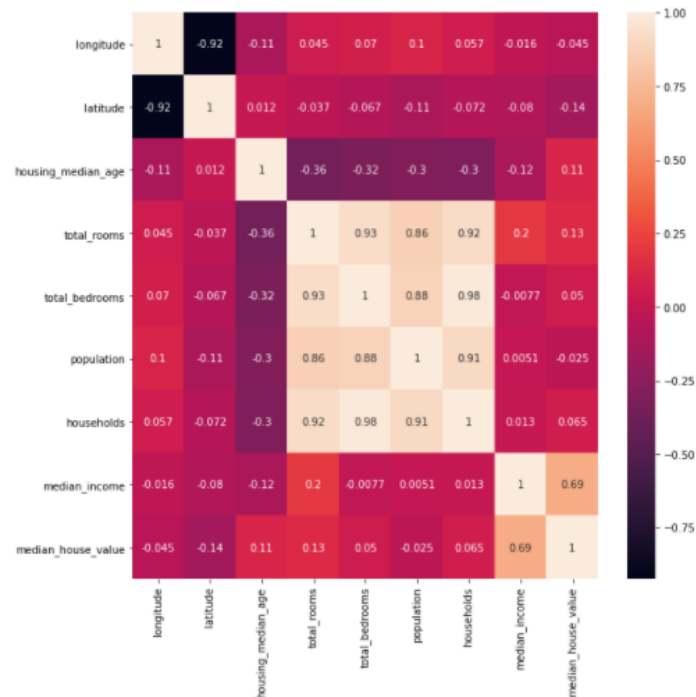


Рис.9. Heatmap-графік.

Бачимо, що найбільша залежність спостерігається між `total_rooms`, `total_bedrooms`, `population` та `households` (більше 0.88), але ці ознаки не корелюють з нашою цільовою ознакою `median_house_value`, окрім загальної кількості кімнат `total_rooms` на рівні 0.2, що все одно є малою величиною. Ця візуалізація лише підтверджує існування залежності `median_house_value` з `median_income`, але тільки на рівні 0.69, що є непоганим, але б хотілось кращої кореляції [1, стор.264-266].

1.3. Знаходження математичного сподівання та дисперсії σ для набору даних, стандартизація даних.

Використовуючи метод `describe()` ми можемо отримати інформацію про середні, мінімальні, максимальні значення, значення квантилів і т.д (див. рис.10), але там не міститься інформація про дисперсію, хоча є про значення її кореня, однак завдання вимагає саме дисперсію, тому використовуючи метод `var()` знаходимо її (див. рис.11) [2, стор.158].

`housing_prices_base.describe()`

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20433.000000	20433.000000	20433.000000	20433.000000	20433.000000	20433.000000	20433.000000	20433.000000	20433.000000
mean	-119.570689	35.633221	28.633094	2636.504233	537.870553	1424.946949	499.433465	3.871162	206864.413155
std	2.003578	2.136348	12.591805	2185.269567	421.385070	1133.208490	382.299226	1.899291	115435.667099
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1450.000000	296.000000	787.000000	280.000000	2.563700	119500.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.536500	179700.000000
75%	-118.010000	37.720000	37.000000	3143.000000	647.000000	1722.000000	604.000000	4.744000	264700.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

Рис.10. Опис датасету.

`housing_prices_base.var()`

```
longitude      4.014324e+00
latitude       4.563981e+00
housing_median_age  1.585536e+02
total_rooms    4.775403e+06
total_bedrooms  1.775654e+05
population     1.284161e+06
households     1.461527e+05
median_income  3.607307e+00
median_house_value 1.332539e+10
dtype: float64
```

Рис.11. Значення дисперсії.

Виконаємо стандартизацію даних і ось тут вже це не просто тому що від нас так хочуть у завданні, а через те, що метод PCA, якій далі буде використовуватись, просто це вимагає робити обов'язково [1, стор.134]. Для цього використовуємо метод `StandardScaler()` [1, стор.70], але перед цим представимо ознаку `ocean_proximity` у числову вигляді, використовуючи метод `LabelEncoder()` [1, стор.113-114]. Вигляд нових даних можна побачити на рис.12.

```
from sklearn.preprocessing import StandardScaler, LabelEncoder

housing_prices_base['ocean_proximity'] =
LabelEncoder().fit_transform(housing_prices_base['ocean_proximity'])

housing_prices_base_normalized=pd.DataFrame(StandardScaler().fit_transform(housing
_prices_base))

housing_prices_base_normalized.columns=housing_prices_base.columns

housing_prices_base_normalized.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-1.327314	1.051717	0.982163	-0.803813	-0.970325	-0.973320	-0.976833	2.345163	2.128819	1.291351
1	-1.322323	1.042355	-0.606210	2.042130	1.348276	0.861339	1.670373	2.332632	1.313626	1.291351
2	-1.332305	1.037674	1.855769	-0.535189	-0.825561	-0.819769	-0.843427	1.782939	1.258183	1.291351
3	-1.337296	1.037674	1.855769	-0.623510	-0.718768	-0.765056	-0.733562	0.932970	1.164622	1.291351
4	-1.337296	1.037674	1.855769	-0.461970	-0.611974	-0.758879	-0.628930	-0.013143	1.172418	1.291351

Рис.12. Вигляд стандартизованого набору даних.

Якщо перевірити математичне сподівання та дисперсію стандартизованого набору даних, то дійсно матимемо розподіл, що є близьким до нормального (див. рис.13 та рис.14).

```
housing_prices_base_normalized.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
count	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04	2.043300e+04
mean	-3.337486e-15	6.980827e-16	-7.584704e-16	2.192776e-16	2.035816e-16	-2.318367e-16	1.299940e-16	3.832505e-16	-1.025841e-16	-2.248267e-15
std	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00	1.000024e+00
min	-2.385447e+00	-1.447937e+00	-2.194584e+00	-1.205804e+00	-1.274093e+00	-1.254828e+00	-1.303810e+00	-1.775054e+00	-1.682139e+00	-8.211765e-01
25%	-1.112692e+00	-7.972780e-01	-8.444662e-01	-5.429688e-01	-5.740034e-01	-5.629702e-01	-5.739976e-01	-6.884113e-01	-7.588419e-01	-8.211765e-01
50%	5.393926e-01	-8.428050e-01	2.913923e-02	-2.331597e-01	-2.441308e-01	-2.285134e-01	-2.365573e-01	-1.762077e-01	-2.353266e-01	-1.170008e-01
75%	7.789899e-01	9.788213e-01	6.844888e-01	2.317829e-01	2.589843e-01	2.621409e-01	2.735268e-01	4.595713e-01	5.010323e-01	-1.170008e-01
max	2.625711e+00	2.956885e+00	1.855769e+00	1.678713e+01	1.401871e+01	3.023088e+01	1.460297e+01	5.859865e+00	2.539456e+00	1.995526e+00

Рис.13. Опис стандартизованого набору даних.

`housing_prices_base_normalized.var()`

```
longitude    1.000049
latitude     1.000049
housing_median_age  1.000049
total_rooms   1.000049
total_bedrooms 1.000049
population   1.000049
households   1.000049
median_income 1.000049
median_house_value 1.000049
ocean_proximity 1.000049
dtype: float64
```

Рис.14. Значення дисперсії стандартизованого набору даних.

Однак, якщо зробити візуалізацію отриманого розподілу (рис.15), то можна побачити, що поведінка стандартизованого набору даних дуже схожа на минулий розподіл, але за числовими даними (з рис.13 та рис.14) близька до нормального.

`housing_prices_base_normalized.hist(bins=50, figsize=(20,10))`

`plt.show()`

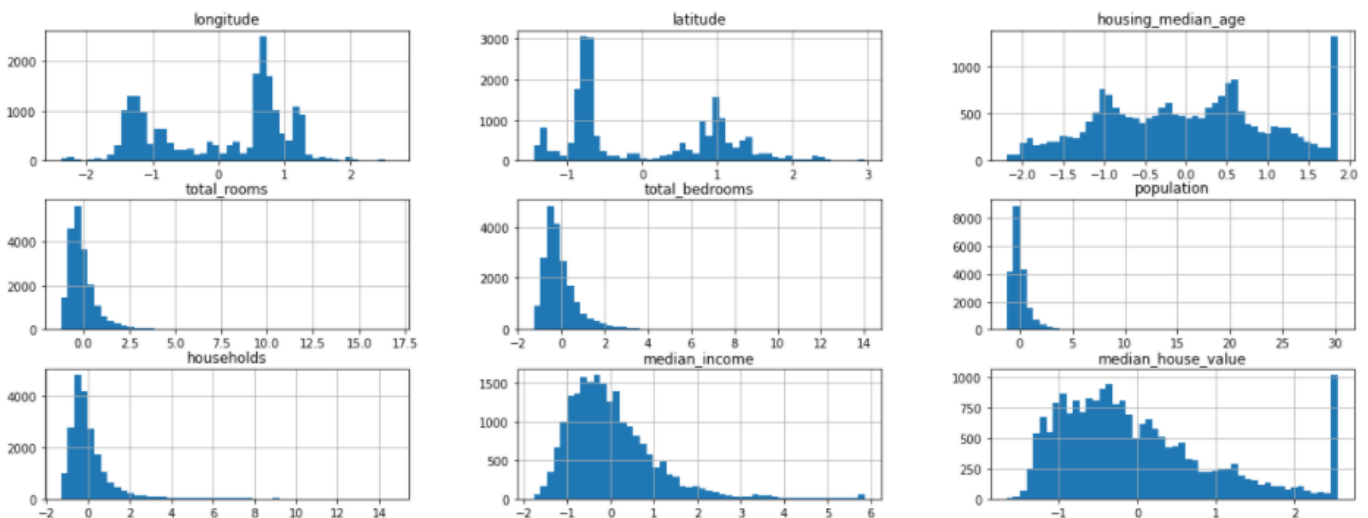


Рис.15. Візуалізація стандартизованого набору даних.

1.4. Визначення інформативності ознак за допомогою моделі Random Forest.

Щоб визначити інформативність ознак даного датасету для прогнозування середньої ціни на житло у Каліфорнії використовую метод `feature_importances_` [1, стор.130-132]. На рис.16 можна побачити, що дійсно найінформативнішою ознакою є `median_income`, тобто середня заробітна плата людей вже заселених будинків того району, де те чи інше житло знаходиться.

```

from sklearn.ensemble import RandomForestRegressor

X=housing_prices_base_normalized.drop('median_house_value',axis = 1)

Y=housing_prices_base_normalized['median_house_value']

Random_Forest_Regressor=RandomForestRegressor(n_estimators=250,random_state=0
).fit(X,Y)

importances = Random_Forest_Regressor.feature_importances_

for index,feature in enumerate(X):

    print(f'{feature} importance: {importances[index]}')

sns.barplot(x=importances,y=X.columns)

```

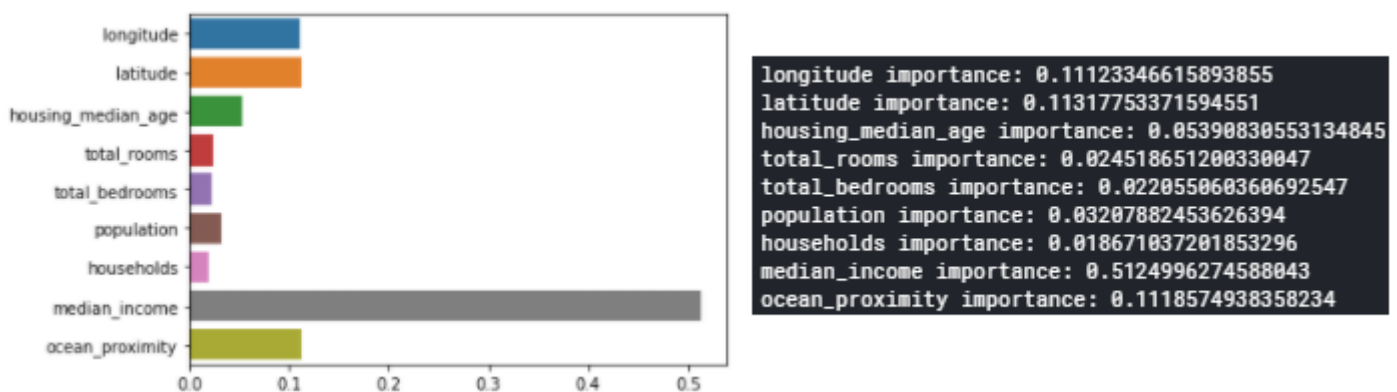


Рис.16. Інформативність ознак.

1.5.1. Зменшення розмірності даних за допомогою методу Principal Components Analysis (PCA) до двох вимірів та візуалізація даних.

Після виконання даного завдання на рисунках 18-21 можна побачити, що розподіл гарнішим не став, а прямої чи оберненої кореляції взагалі нема, що зводить успіх прогнозу до нуля, тому з отриманим набором даних шляхом зменшення розмірності до двох вимірів працювати не буду [1, стор.140-141].

```

from sklearn.decomposition import PCA

X_PCA=PCA(n_components=2).fit_transform(X)

```

```
housing_prices_base_normalized_PCA=pd.DataFrame(data=X_PCA,columns=['PC1','PC2'])
```

```
housing_prices_base_normalized_PCA['median_house_value']=housing_prices_base_normalized['median_house_value']
```

```
housing_prices_base_normalized_PCA.head()
```

	PC1	PC2	median_house_value
0	-2.154315	1.702369	2.128819
1	2.866326	2.304909	1.313626
2	-2.032445	1.792476	1.258183
3	-1.982123	1.833709	1.164622
4	-1.840613	1.893802	1.172418

Рис.17. Вигляд даних, після зменшення розмірності предикторів до 2х вимірів.

```
fig = plt.figure(figsize = (20, 10))
```

```
ax_1 = fig.add_subplot(1, 2, 1)
```

```
ax_2 = fig.add_subplot(1, 2, 2)
```

```
sns.lineplot(data=housing_prices_base_normalized_PCA,x='PC1',y='median_house_value',ax=ax_1)
```

```
sns.lineplot(data=housing_prices_base_normalized_PCA,x='PC2',y='median_house_value',ax=ax_2)
```

```
plt.show()
```

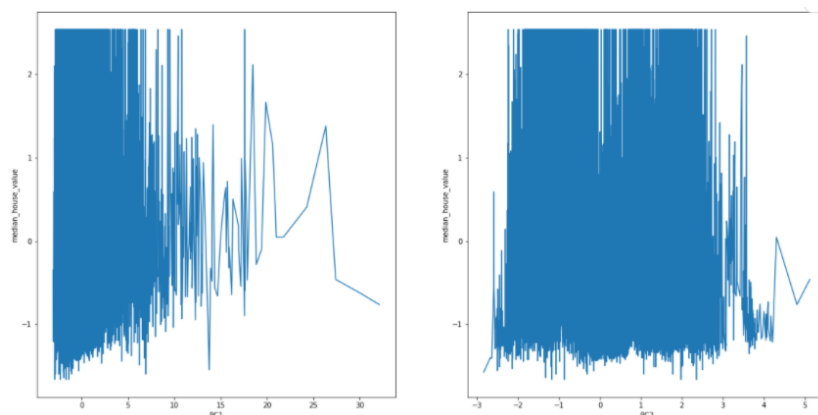


Рис.18. Лінійні залежності, після зменшення розмірності предикторів до 2х вимірів.


```
fig = plt.figure(figsize = (20, 10))

ax_1 = fig.add_subplot(1, 2, 1)

ax_2 = fig.add_subplot(1, 2, 2)

sns.barplot(data=housing_prices_base_normalized_PCA,x='PC1',y='median_house_value',ax=ax_1)

sns.barplot(data=housing_prices_base_normalized_PCA,x='PC2',y='median_house_value',ax=ax_2)

plt.show()
```

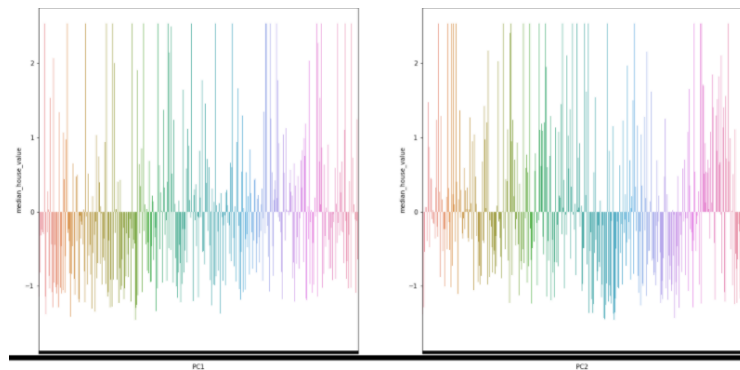


Рис.19. BAR-графіки, після зменшення розмірності предикторів до 2х вимірів.

```
fig = plt.figure(figsize=(10,10))

sns.heatmap(data=housing_prices_base_normalized_PCA.corr(),annot=True)
```

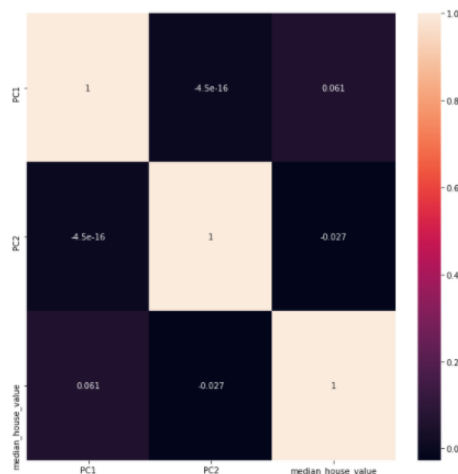


Рис.20. Heatmap-графік, після зменшення розмірності предикторів до 2х вимірів.

```
housing_prices_base_normalized_PCA.hist(bins=50, figsize=(20,10))

plt.show()
```

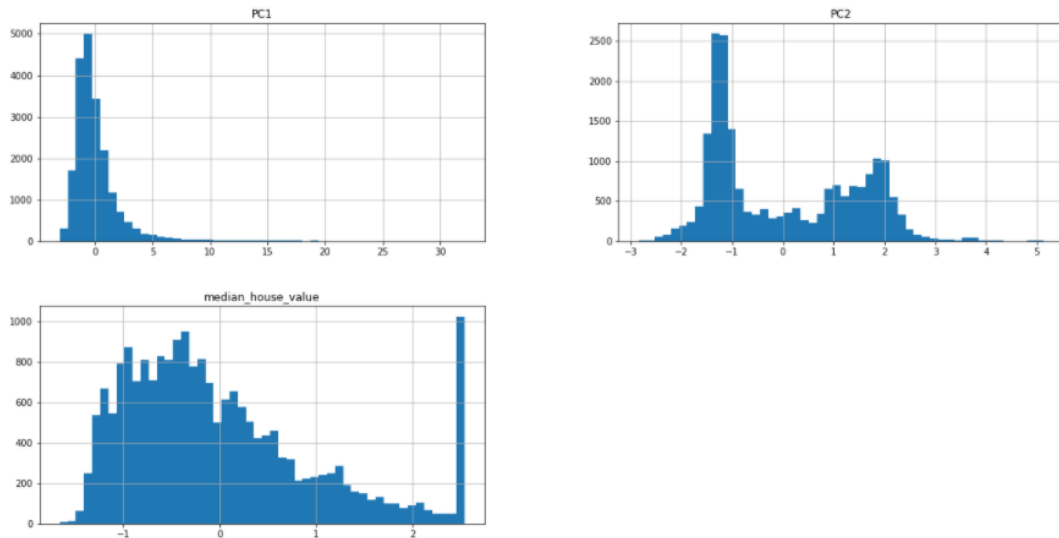


Рис.21. Гістограми, після зменшення розмірності предикторів до 2х вимірів.

1.5.2. Знаходження відсотків дисперсії для власних векторів та знаходження необхідної кількості власних векторів, щоб залишилося більше 90% дисперсії.

Використовуючи методи `explained_variance_` та `explained_variance_ratio_` було знайдено власні числа та відсотки дисперсії власних векторів коваріаційної матриці [інтернет-ресурс 1], що була отримана при виклику методу PCA (див. рис.22 та рис.23).

```
pca=PCA(n_components=None).fit(X)
```

```
pca.explained_variance_
```

```
array([3.91241513, 2.02163819, 1.07544632, 0.93615543, 0.7686712 ,
       0.14713054, 0.0794061 , 0.04474594, 0.01483163])
```

Рис.22. Власні числа коваріаційної матриці.

```
pca.explained_variance_ratio_
```

```
array([0.43469152, 0.22461547, 0.11948819, 0.10401218, 0.08540373,
       0.01634704, 0.00882247, 0.00497153, 0.00164788])
```

Рис.23. Відсотки дисперсії власних векторів.

Підрахунок кількості власних векторів, щоб значення дисперсії було більше за 90 відсотків, з рис. 24 говорить, що їх потрібно хоча б п'ять.

```
summ=0.0

for index,ratio in enumerate(pca.explained_variance_ratio_):

    summ+=ratio

    print(f'{index+1} {summ}')
```

```
1 0.4346915166853468
2 0.6593069894273401
3 0.778795176758534
4 0.8828073563671667
5 0.9682110881019435
6 0.9845581259839083
7 0.9933805942079276
8 0.9983521219438354
9 1.0
```

Рис.24. Підрахунок відсотків дисперсії для різної кількості власних векторів.

```
X_PCA=PCA(n_components=5).fit_transform(X)

housing_prices_PCA_base=pd.DataFrame(data=X_PCA,columns=['PC1','PC2','PC3','PC4','PC5'])

housing_prices_PCA_base['median_house_value']=housing_prices_base_normalized['median_house_value']

housing_prices_PCA_base.head()
```

	PC1	PC2	PC3	PC4	PC5	median_house_value
0	-2.154315	1.702369	1.858370	1.701211	0.699460	2.128819
1	2.866326	2.304909	1.983678	1.450059	0.219830	1.313626
2	-2.032445	1.792476	0.975684	1.894335	1.257428	1.258183
3	-1.982123	1.833709	0.207793	1.575422	1.023919	1.164622
4	-1.840613	1.893802	-0.615396	1.241132	0.778095	1.172418

Рис.25. Датафрейм, отриманий після застосування методу PCA.

1.6. Оцінка залежності між набором даних та набором предикторів за допомогою коефіцієнту Пірсона.

Для оцінки залежностей будуюмо heatmap-графіки (див. рис.26) та бачимо, що після застосування методу опорних векторів немає ознаки, з якою кореляція була більше за 0.69, тому задачу регресії у розділі 2 проводимо з просто стандартизованим датасетом і початковою розмірністю.

```
fig = plt.figure(figsize = (20, 10))
```

```
ax_1 = fig.add_subplot(1, 2, 1)
```

```
ax_2 = fig.add_subplot(1, 2, 2)
```

```
sns.heatmap(data=housing_prices_PCA_base.corr(method='pearson'),annot=True,  
ax=ax_1)
```

```
sns.heatmap(data=housing_prices_base_normalized.corr(method='pearson'),annot=True,  
ax=ax_2)
```

```
plt.show()
```

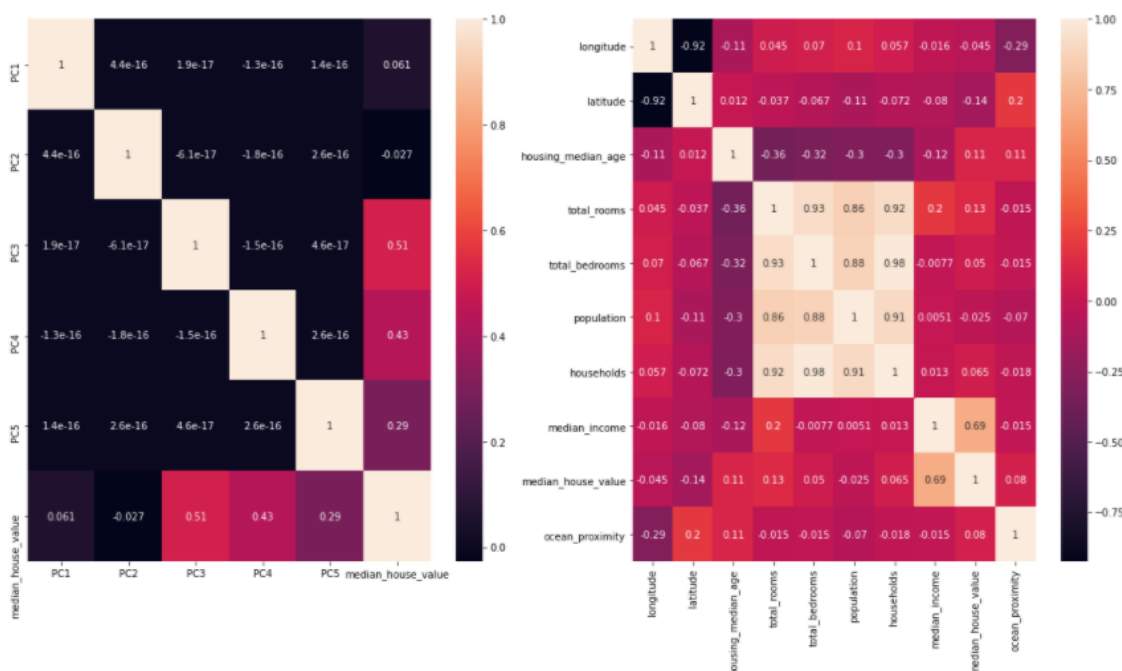


Рис.26. Порівняння heatmap-графіків (зліва – для датафрейму, отриманого після використання методу PCA, справа – для стандартизованого вхідного датафрейму).

1.7. Визначення чи розподіл даних є подібним до нормального розподілу.

Для перевірки того, що розподіл даних є подібним до нормального використовуємо правила трьох, двох та одного сігма [інтернет-ресурс 8], адже математиками було отримано, що якщо розподіл є подібним до нормального, то повинні виконуватись наступні твердження:

$$\Pr(\mu - 1\sigma \leq X \leq \mu + 1\sigma) \approx 0.6827$$

$$\Pr(\mu - 2\sigma \leq X \leq \mu + 2\sigma) \approx 0.9545$$

$$\Pr(\mu - 3\sigma \leq X \leq \mu + 3\sigma) \approx 0.9973$$

На рис.27 бачимо, що наші ознаки подібні до нормально розподілених, але не є нормально розподіленими.

```
def find_probabilities(df):
```

```
    for index, feature in enumerate(df.columns):
```

```
        print(f'\n{index+1})feature: {feature}')
```

```
        m=df[feature].mean()
```

```
        s=df[feature].std()
```

```
        valids=[0,0,0]
```

```
        for j in df[feature].values:
```

```
            if (m-s)<=j and j<=(m+s):
```

```
                valids[0]+=1
```

```
            if (m-2*s)<=j and j<=(m+2*s):
```

```
                valids[1]+=1
```

```
            if (m-3*s)<=j and j<=(m+3*s):
```

```
valids[2]+=1
```

```
print(f'Pr( $\mu - 1\sigma \leq \{feature\} \leq \mu + 1\sigma$ ) =', valids[0] / df.shape[0])
```

```
print(f'Pr( $\mu - 2\sigma \leq \{feature\} \leq \mu + 2\sigma$ ) =', valids[1] / df.shape[0])
```

```
print(f'Pr( $\mu - 3\sigma \leq \{feature\} \leq \mu + 3\sigma$ ) =', valids[2] / df.shape[0])
```

```
find_probabilities(housing_prices_PCA_base)
```

```
1)feature: PC1
Pr( $\mu - 1\sigma \leq PC1 \leq \mu + 1\sigma$ ) = 0.850095433856996
Pr( $\mu - 2\sigma \leq PC1 \leq \mu + 2\sigma$ ) = 0.9624626829148926
Pr( $\mu - 3\sigma \leq PC1 \leq \mu + 3\sigma$ ) = 0.9827240248617433

2)feature: PC2
Pr( $\mu - 1\sigma \leq PC2 \leq \mu + 1\sigma$ ) = 0.6397983653893212
Pr( $\mu - 2\sigma \leq PC2 \leq \mu + 2\sigma$ ) = 0.990994959134733
Pr( $\mu - 3\sigma \leq PC2 \leq \mu + 3\sigma$ ) = 0.9998531786815446

3)feature: PC3
Pr( $\mu - 1\sigma \leq PC3 \leq \mu + 1\sigma$ ) = 0.7197181030685655
Pr( $\mu - 2\sigma \leq PC3 \leq \mu + 2\sigma$ ) = 0.9588900308324769
Pr( $\mu - 3\sigma \leq PC3 \leq \mu + 3\sigma$ ) = 0.9880585327656243

4)feature: PC4
Pr( $\mu - 1\sigma \leq PC4 \leq \mu + 1\sigma$ ) = 0.6927519209122498
Pr( $\mu - 2\sigma \leq PC4 \leq \mu + 2\sigma$ ) = 0.9500318112856654
Pr( $\mu - 3\sigma \leq PC4 \leq \mu + 3\sigma$ ) = 0.9955953604463368

5)feature: PC5
Pr( $\mu - 1\sigma \leq PC5 \leq \mu + 1\sigma$ ) = 0.6929966231096756
Pr( $\mu - 2\sigma \leq PC5 \leq \mu + 2\sigma$ ) = 0.9548769147946948
Pr( $\mu - 3\sigma \leq PC5 \leq \mu + 3\sigma$ ) = 0.9957911222042773

6)feature: median_house_value
Pr( $\mu - 1\sigma \leq median\_house\_value \leq \mu + 1\sigma$ ) = 0.7019527235354573
Pr( $\mu - 2\sigma \leq median\_house\_value \leq \mu + 2\sigma$ ) = 0.9328537170263789
Pr( $\mu - 3\sigma \leq median\_house\_value \leq \mu + 3\sigma$ ) = 1.0
```

Рис.27. Результат застосування правила трьох сігма до датафрейму, отриманого після використання методу PCA

1.8. Видалення з набору всіх даних, для яких $\Pr(\mu - 3\sigma \leq X \leq \mu + 3\sigma) > 0.9973$.

Для виконання цього завдання нам треба просто скористатись методом `drop()` та видалити колонку PC2. Колонку `median_house_value` видаляти не будемо, адже цю колонку ми хочемо прогнозувати. Результат видалення можна побачити на рис.28.

```
housing_prices_PCA_base=housing_prices_PCA_base.drop(columns=['PC2'])
```

```
housing_prices_PCA_base.head()
```

	PC1	PC3	PC4	PC5	median_house_value
0	-2.154315	1.858370	1.701211	0.699460	2.128819
1	2.866326	1.983678	1.450059	0.219830	1.313626
2	-2.032445	0.975684	1.894335	1.257428	1.258183
3	-1.982123	0.207793	1.575422	1.023919	1.164622
4	-1.840613	-0.615396	1.241132	0.778095	1.172418

Рис.28. Дані після видалення ознак, для яких ймовірність трьох сігма є більшою за 0.997.

Якщо ми пункти 7 та 8 виконаємо для вхідного стандартизованого датасету, то він перестане мати якісь ознаки, адже його розподіл дуже схожий на нормальний і ми все видалимо.

1.9. Проведення тесту Колмогорова-Смірнова.

Для проведення тесту Колмогорова-Смірнова використовуємо метод `kstest()` з бібліотеки `scipy` та обраховуємо статистику для кожної ознаки датасету [інтернет-ресурс 7] та перевіряємо чи менша ця статистика за критичне значення з рівнем довіри альфа чи ні. Ми перевіряємо справедливність гіпотези про те, що ознака розподілена за нормальним розподілом. Якщо значення статистики Колмогорова-Смірнова більше за критичне значення, то ми кажемо, що гіпотеза не відхиляється, інакше – відхиляється.

Для даного датасету маємо, що гіпотеза про нормальний розподіл на жаль відкидається, якщо використовувати рівень довіри 0.05 (з рівням довіри 0.1 ситуація аналогічна). Повідомлення про такий результат можна побачити на рис.29, а для наочності на рис.30 можна побачити гістограми розподілів ознак, за якими видно, що розподіл не є нормальним.

```
from scipy.stats import kstest
```

```
def check_hypotesis(df,feature,statistic,alpha):
```

```
    n=df.shape[0]
```

```
    critical_value=(1/np.sqrt(n))*np.sqrt(-np.log(alpha/2))
```

```

print(f'feature = {feature}')

if statistic < critical_value:

    print(f'statistic {statistic} < critical_value {critical_value} => the hypothesis is
    not rejected at the alpha {alpha} significance level')

else:

    print(f'statistic {statistic} >= critical_value {critical_value} => the hypothesis
    is rejected at the alpha {alpha} significance level')

for feature in housing_prices_PCA_base.columns:

    statistic = kstest(housing_prices_PCA_base[feature], 'norm').statistic

    check_hypotesis(housing_prices_PCA_base, feature, statistic, 0.05)

```

```

feature = PC1
statistic 0.17549208976105157 >= critical_value 0.0134363455482532 => the hypothesis is rejected at the alpha 0.05 significance level
feature = PC3
statistic 0.061965845721411394 >= critical_value 0.0134363455482532 => the hypothesis is rejected at the alpha 0.05 significance level
feature = PC4
statistic 0.10665821316214641 >= critical_value 0.0134363455482532 => the hypothesis is rejected at the alpha 0.05 significance level
feature = PC5
statistic 0.05166065392507635 >= critical_value 0.0134363455482532 => the hypothesis is rejected at the alpha 0.05 significance level
feature = median_house_value
statistic 0.10297613682619272 >= critical_value 0.0134363455482532 => the hypothesis is rejected at the alpha 0.05 significance level

```

Рис.29. Результати тесту Колмогорова-Смірнова про нормальність розподілу.

```
housing_prices_PCA_base.hist(bins=200, figsize=(20,10))
```

```
plt.show()
```

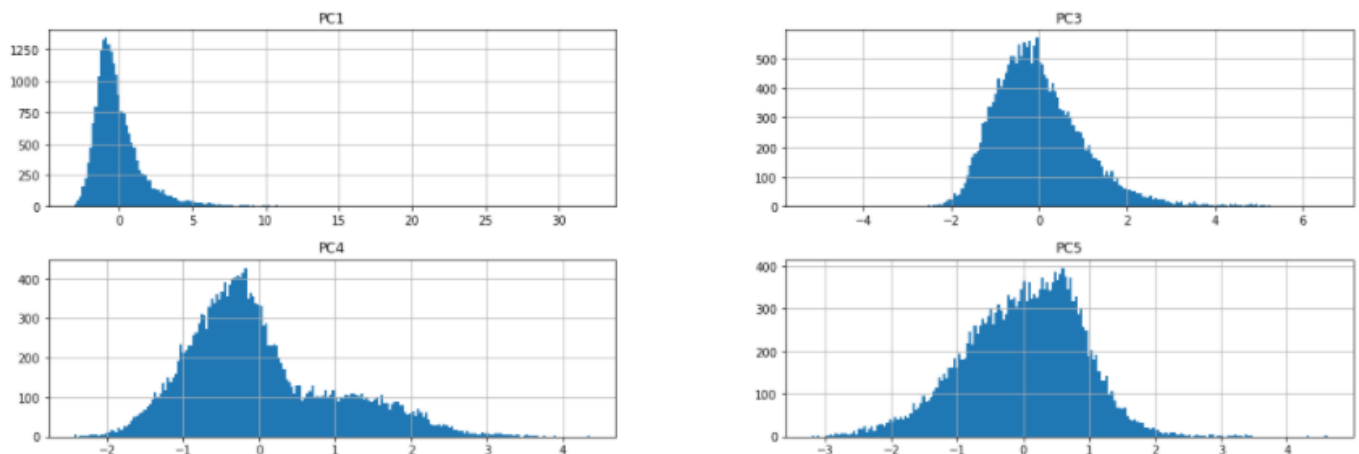


Рис.30. Гістограма розподілів ознак.

РОЗДІЛ 2

РЕАЛІЗАЦІЯ МОДЕЛІ

Після попередньої обробки даних та аналізу даних, що були зроблені у першому розділі, можна перейти до вирішення поставленої задачі: побудови моделі, що передбачає середню вартість житла у Каліфорнії. Зрозуміло, що дана задача є задачею регресії, для розв'язку якої я можу використовувати декілька методів, починаючи з найпростішої моделі лінійної регресії, закінчуючи деревами рішень [1, стор.277, 285-287].

На рис.31 можна побачити точності на тренувальних та тестових вибірках після застосування різних методів до стандартизованого датасету. З курсу «Методів машинного навчання» знаємо, що модель вважається гарною не тільки коли у неї точність прогнозу висока, а й коли точності прогнозу на тренувальних та тестових вибірках майже однакові, адже інакше наша модель може, наприклад, просто перевчитись. У моєму випадку гарними моделями є SVR [інтернет-ресурс 6] та MLPRegressor [інтернет-ресурс 2], тому при ансамблюванні [інтернет-ресурс 4] я використовую саме їх і отримую точність 0.76.

Варто зазначити, що оскільки дані не є нормально розподіленими, то вчені у цьому випадку рекомендують використовувати достатню велику кількість спостережень для отримання найточніших результатів. Даний датасет, після видалення усіх пропущених значень, містить 20433 спостереження і я вважаю це достатньо непоганою кількістю.

```
from sklearn.model_selection import train_test_split

X = housing_prices_base_normalized_k_means.drop('median_house_value',axis = 1)

Y = housing_prices_base_normalized_k_means['median_house_value']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.5, random_state=0)

from sklearn.linear_model import Lasso, Ridge, ElasticNet, TweedieRegressor
```

```

from sklearn.neural_network import MLPRegressor

from sklearn.neighbors import KNeighborsRegressor

from sklearn.svm import SVR

from sklearn.ensemble import RandomForestRegressor

from sklearn.ensemble import VotingRegressor

from sklearn.linear_model import LinearRegression

regression_models=[LinearRegression(),Lasso(),Ridge(),ElasticNet(),TweedieRegressor()
,MLPRegressor(),KNeighborsRegressor(),SVR(),RandomForestRegressor()]

Accuracy_test=[]

Accuracy_train=[]

for index,model in enumerate(regression_models):

    print(f'{index}. {model}')

    reg_model=model

    reg_model.fit(X_train,Y_train)

    Y_predict_train=reg_model.predict(X_train)

    Y_predict_test=reg_model.predict(X_test)

    Accuracy_train.append(reg_model.score(X_train, Y_train))

    Accuracy_test.append(reg_model.score(X_test, Y_test))

models_ana_their_accuracy=pd.DataFrame(data={'Model':regression_models,'Accuracy
_train':Accuracy_train,'Accuracy_test':Accuracy_test})

models_ana_their_accuracy

```

	Model	Accuracy_train	Accuracy_test
0	LinearRegression()	0.630707	0.642433
1	Lasso()	0.000000	-0.000017
2	Ridge()	0.630707	0.642434
3	ElasticNet()	0.153260	0.153791
4	TweedieRegressor()	0.407907	0.412410
5	MLPRegressor()	0.784156	0.765374
6	KNeighborsRegressor()	0.797699	0.702728
7	SVR()	0.723799	0.721557
8	(DecisionTreeRegressor(max_features='auto', ra...	0.971706	0.806814

Рис.31. Точності на тренувальних та тестових вибірках для різних моделей.

```
voting=VotingRegressor([('SVR', regression_models[7]),('MLPRegressor',
regression_models[5])])
```

```
voting=voting.fit(X_train,Y_train)
```

```
Y_pred=voting.predict(X_test)
```

```
print('Accuracy: ', voting.score(X_test, Y_test))
```

РОЗДІЛ 3

АНАЛІЗ РЕЗУЛЬТАТІВ

1.1. Вплив відновлення пропущених значень порівняно з видаленням їх з вибірки.

У розглянутому датасеті було видалено 207 спостережень, у яких були відсутні записи щодо тих чи інших ознак, що складає приблизно 1 відсоток від усього датасету, тому можна вважати, що видалення ніяк не вплинуло на подальший аналіз та результати тестів.

1.2. Аналіз побудованих графіків та гістограм для різних ознак.

Для вхідного датасету розподіл таких ознак як total_rooms, total_bedrooms, population та households був дуже схожий на нормальний, але збільшивши кількість бінів та масштаб у гістограмах можна було побачити, що він таким не є. Ознака, що

була вибрана цільовою, `median_house_value` у районі 500000 мала приблизно 1000 викидів, її видалення не сильно впливало на точність, тому ці викиди я не прибирала. Також було отримано, що ціни на житло з виходом до океану чи затоки вище у два рази, ніж ціни на житло без виходу до них, а найвищі ціни на житло, що знаходиться на островах. З графіків лінійної залежності та BAR-графіків між набором даних та `median_house_value` яскравих залежностей між набором даних та `median_house_value` спостерігати не довелось, окрім залежності `median_house_value` від `median_income`, тобто залежності середньої вартості житла від середньої заробітної плати людей вже заселених будинків того району, де це житло знаходиться. З heatmap-графіків найбільша залежність спостерігалась між `total_rooms`, `total_bedrooms`, `population` та `households` (більше 0.88), а також залежність `median_house_value` з `median_income` на рівні 0.69.

1.3. Аналіз отриманих даних після застосування алгоритму зменшення розмірності.

Після зменшення розмірності датасету до двох вимірів розподіл ознак не став гарнішим, а пряма чи обернена кореляція була відсутня, тому з отриманим набором даних я не працювала.

Після зменшення розмірності датасету до п'яти вимірів з heatmap-графіків можна було побачити, що немає ознаки, з якою кореляція була більше за 0.69, тому задачу регресії у розділі 2 проводила з просто стандартизованим датасетом і початковою розмірністю.

1.4. Аналіз впливу зменшення розмірності даних на точність моделі.

У розділі 2 я подала найкращі з отриманих мною результатів (прогнозування за стандартизованим датасетом). Якщо вирішувати задачу регресії після зменшення розмірності датасету, то точність буде 0.618, що є очікуваним, адже коефіцієнт кореляції предикторів датасету після зменшення розмірності і цільової ознаки у найкращому випадку 0.51.

1.5. Аналіз залежності між предикторами та досліджуваною величиною.

Після зменшення розмірності вхідного датасету, використовуючи метод опорних векторів найбільша кореляція спостерігалась між `median_house_value` та PC3, PC4, PC5 (коефіцієнти відповідно 0.51, 0.43, 0.29), однак для побудови точного прогнозу хотілось мати набагато кращі коефіцієнти, тому у розділі 2 я працювала з просто стандартизованими вхідними даними.

Варто зазначити, що була спроба підвищити точність, шляхом збільшення розмірності, тобто введення нових ознак: попередньої кластеризації [1, стор.289-292] даних за ознаками `median_income`, `population`, `households` та `total_rooms`, але вона не давала кращого результату, тому не наведена у даній роботі.

1.6. Аналіз розподілу вхідних даних, аналіз викидів, після переведення його до нормального, за правилом 68-95-99.7.

Якщо ми говоримо про розподіл даних після стандартизації, то можна сказати за рис.15, що розподіл не змінився, а за числовими обрахунками (значення математичного очікування та дисперсії дивитись на рисунках 13 та 14) ми маємо розподіл, що є близьким до нормального. Використовуючи правило 68-95-99.7 [інтернет-ресурс 8] можна побачити, що розподіл є близьким до нормального (див. рис.32).

find_probabilities(housing_prices_base_normalized)

1.7. Аналіз результатів тесту Колмогорова-Смірнова.

Якщо ми говоримо про результати тесту Колмогорова-Смірнова для стандартизованих даних, то ми бачимо, що гіпотеза про нормальність розподілу ознак відкидається навіть з рівнем довіри 0.1, що ще раз нам говорить про те, що стандартизація не робить дані нормально розподіленими, хоча інколи трапляються і такі випадки.

```

1)feature: longitude
Pr( $\mu - 1\sigma \leq \text{longitude} \leq \mu + 1\sigma$ ) = 0.5881172612930065
Pr( $\mu - 2\sigma \leq \text{longitude} \leq \mu + 2\sigma$ ) = 0.9873733666128321
Pr( $\mu - 3\sigma \leq \text{longitude} \leq \mu + 3\sigma$ ) = 1.0

2)feature: latitude
Pr( $\mu - 1\sigma \leq \text{latitude} \leq \mu + 1\sigma$ ) = 0.68780893652425
Pr( $\mu - 2\sigma \leq \text{latitude} \leq \mu + 2\sigma$ ) = 0.979298194097783
Pr( $\mu - 3\sigma \leq \text{latitude} \leq \mu + 3\sigma$ ) = 1.0

3)feature: housing_median_age
Pr( $\mu - 1\sigma \leq \text{housing\_median\_age} \leq \mu + 1\sigma$ ) = 0.6296187539764108
Pr( $\mu - 2\sigma \leq \text{housing\_median\_age} \leq \mu + 2\sigma$ ) = 0.9939313855038419
Pr( $\mu - 3\sigma \leq \text{housing\_median\_age} \leq \mu + 3\sigma$ ) = 1.0

4)feature: total_rooms
Pr( $\mu - 1\sigma \leq \text{total\_rooms} \leq \mu + 1\sigma$ ) = 0.8738315470072922
Pr( $\mu - 2\sigma \leq \text{total\_rooms} \leq \mu + 2\sigma$ ) = 0.9640777174179024
Pr( $\mu - 3\sigma \leq \text{total\_rooms} \leq \mu + 3\sigma$ ) = 0.9818430969510106

5)feature: total_bedrooms
Pr( $\mu - 1\sigma \leq \text{total\_bedrooms} \leq \mu + 1\sigma$ ) = 0.8602750452699065
Pr( $\mu - 2\sigma \leq \text{total\_bedrooms} \leq \mu + 2\sigma$ ) = 0.9604561248960015
Pr( $\mu - 3\sigma \leq \text{total\_bedrooms} \leq \mu + 3\sigma$ ) = 0.981989918269466

6)feature: population
Pr( $\mu - 1\sigma \leq \text{population} \leq \mu + 1\sigma$ ) = 0.8612049136201243
Pr( $\mu - 2\sigma \leq \text{population} \leq \mu + 2\sigma$ ) = 0.9633925512651104
Pr( $\mu - 3\sigma \leq \text{population} \leq \mu + 3\sigma$ ) = 0.9834091910145353

7)feature: households
Pr( $\mu - 1\sigma \leq \text{households} \leq \mu + 1\sigma$ ) = 0.8520530514364019
Pr( $\mu - 2\sigma \leq \text{households} \leq \mu + 2\sigma$ ) = 0.9606518866539422
Pr( $\mu - 3\sigma \leq \text{households} \leq \mu + 3\sigma$ ) = 0.9822835609063769

8)feature: median_income
Pr( $\mu - 1\sigma \leq \text{median\_income} \leq \mu + 1\sigma$ ) = 0.7588215142171977
Pr( $\mu - 2\sigma \leq \text{median\_income} \leq \mu + 2\sigma$ ) = 0.9588410903929917
Pr( $\mu - 3\sigma \leq \text{median\_income} \leq \mu + 3\sigma$ ) = 0.9833602505750502

9)feature: median_house_value
Pr( $\mu - 1\sigma \leq \text{median\_house\_value} \leq \mu + 1\sigma$ ) = 0.7019527235354573
Pr( $\mu - 2\sigma \leq \text{median\_house\_value} \leq \mu + 2\sigma$ ) = 0.9328537170263789
Pr( $\mu - 3\sigma \leq \text{median\_house\_value} \leq \mu + 3\sigma$ ) = 1.0

10)feature: ocean_proximity
Pr( $\mu - 1\sigma \leq \text{ocean\_proximity} \leq \mu + 1\sigma$ ) = 0.760289727401752
Pr( $\mu - 2\sigma \leq \text{ocean\_proximity} \leq \mu + 2\sigma$ ) = 1.0
Pr( $\mu - 3\sigma \leq \text{ocean\_proximity} \leq \mu + 3\sigma$ ) = 1.0

```

Рис.32. Результат застосування правила трьох сігма до стандартизованого датафрейму

1.8. Як підрахунок різних точностей описує поведінку навчених моделей?

Для підрахунку точностей прогнозу різних моделей використовувався метод `score()` для тренувальної та тестової вибірок. Моделі, що розглядалися, вважалися гарними, якщо точність прогнозу на тренувальній та тестовій вибірках були близькими, а також складали більше 70 відсотків. На рис.31 можна побачити конкретні значення точностей для кожної з моделей.

1.9. Вибір оптимальних параметрів моделі, їх обґрунтування.

Параметри моделей, що були розглянуті, були встановлені за замовчуванням. Були спроби корегування параметрів моделей з метою покращити точність, але вони не були вдалими, тому я їх не наводила.

1.10. Оцінка помилок на початковій та тестовій вибірках.

На рис.31 можна побачити конкретні значення точностей для кожної з моделей на тренувальній та тестовій вибірках, серед яких найкращі результати показали моделі SVR та MLPRegressor з точностями 0.72 та 0.77 відповідно.

1.11. Вибір найкращих моделей для ансамблювання на основі їх точностей та вибір всіх моделей для ансамблювання.

З рис.31 бачимо, що найкращий результат дали моделі SVR та MLPRegressor з точностями 0.72 та 0.77, тому для ансамблювання були використані саме вони. RandomForestRegressor не був обраний у якості найкращої моделі, адже різниця точностей на тренувальній та тестовій вибірках склала 0.16 (точність на тренувальній вибірці 0.97, а на тестовій 0.81), що свідчить про те, що наша модель перенавчилась. Була зроблена спроба покращити RandomForestRegressor, вибираючи інші параметри, але точність складала приблизно 0.63.

Вибір усіх моделей для ансамблювання не має сенсу, адже серед них є дуже погані моделі (Lasso, ElasticNet, TweedieRegressor), у яких точність менша 0.5.

1.12. Порівняння точності ансамблю та найкращої моделі.

Точність ансамблю з найкращих моделей (SVR та MLPRegressor) склала 0.76, а точність найкращої моделі склала 0.77. Очевидно, що різниця точностей не є дуже великою, проте я б радила використовувати ансамбль з моделей (SVR та MLPRegressor), адже різниця точностей після застосування SVR моделі на тренувальній та тестовій вибірках менша, ніж коли ми використовуємо MLPRegressor, а точність самого MLPRegressor вища за точність SVR.

ВИСНОВОК

У даній роботі, яка складається з трьох розділів, я проаналізувала обраний датасет з kaggle та побудувала модель, що передбачає середню вартість житла у Каліфорнії, тобто median_house_value. У першому розділі я отримала, що розподіли вхідних ознак не є нормальними, а після стандартизації лише близькі до нормальних, тому для кращого прогнозу за такими даними виникала необхідність у великій кількості спостережень. На щастя кількість спостережень обраного датасету була 20640, а після видалення пропущених значень 20433, що я вважаю непоганою кількістю. Звичайно, що маючи більшу кількість спостережень, точність могла б бути кращою, але і час роботи моєї програми був довшим. Використовуючи метод опорних векторів оцінку покращити не вийшло, тому у другому розділі я навела прогноз лише за стандартизованими даними (без зменшення розмірності). Точність такого прогнозу дорівнювала 0.76 для ансамблю моделей SVR та MLPRegressor для тестової вибірки, а для тренувальної вибірки точність дорівнювала 0.77, тобто різниця точностей є невеликою, що дозволяє сказати, що отримана модель не перенавчилась. У третьому розділі я надала більш детальний аналіз перших двох розділів. Варто зазначити, що програмну реалізацію цієї роботи можна знайти на kaggle [інтернет-ресурс 9].

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. **Рашка С.** Python и машинное обучение / пер. с англ. А. В. Логунова. – М.: ДМК Пресс, 2017. – 418 с.: ил.
2. **Уэс Маккинли.** Python и анализ данных / Пер. с англ. Слинкин А. А. – М.: ДМК Пресс, 2015. – 482 с.: ил.

ІНТЕРНЕТ-РЕСУРСИ

1. www.scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html#sklearn.decomposition.PCA
2. www.scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html
3. www.scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html
4. www.scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingRegressor.html

5. www.scikit-learn.org/stable/modules/generated/sklearn.linear_model.TweedieRegressor.html
6. www.educba.com/support-vector-regression
7. www.docs.scipy.org/doc/scipy/reference/generated/scipy.stats.kstest.html
8. www.uk.wikipedia.org/wiki/Правило_68-95-99,7
9. www.kaggle.com/anastasiazvychnaya/california-housing-prices